1. 记事本实现--QMainWindow

1.1 QMainWindow与QWidget的区别

1. QWidget是QMainWindow的父类
2. QMainWindow在QWidget框架的基础上，新增了 菜单栏、工具栏、状态栏等内容！
3. QWidget是QT中所有与用户交互类的基类

总结：

QtMainWindow是一个主窗口类，它提供了一些额外的功能，如菜单栏、工具栏、状态栏等，可以方便地创建一个完整的应用程序窗口。

而QtWidget是一个基本的窗口类，它只提供了最基本的窗口功能，如大小调整、移动等，需要自己添加其他的控件和功能。

1.2 功能列表

新建、打开、保存、另存为、字体设置、颜色设置、复制、粘贴、剪切、恢复、撤销等

1.3 实现思路

1.3.1 界面设计

```
《1》给记事本窗口设置图标
《2》给记事本窗口设置标题
《3》给记事本添加菜单栏、工具栏、状态栏
《4》给记事本菜单栏中添加菜单
《5》给记事本菜单栏中的菜单添加动作(选项)
《6》给记事本添加文本编辑区域
```

1.3.2 菜单栏业务实现

选择菜单下的动作或者选项，做对应的业务处理！

思考： 如何知道哪个动作被激活选择了呢？

查找： 动作有对应的信号 triggered()

实现： 信号与槽函数关联，槽函数中做业务处理

1.3.2.1 新建文件

根据windows上记事本的新建动作，

分析：

```
如果当前打开的文件文本内容有被修改：
    弹出消息提示框，是否要对更改进行保存
        保存：
            如果此时的文件名为 无标题-记事本，
                要进行另存为
            否则：
                进行保存 (打开原来的文件，把文本框中的内容写入到文件中，文件关闭)
```

华清远见元宇宙实验中心 yyzlab.com.cn

```
                文本区域清空
                窗口标题变成了无标题-记事本
            不保存：
                文本区域清空
                窗口标题变成了无标题-记事本
            取消：
                什么都不做
如果当前打开的文件文本内容没有被修改：
    文本区域清空
    窗口标题变成了无标题-记事本
```

注意:

当文本内容有修改的时候， 窗口标题要变成 *+ 文件名

思考： 如何知道文本内容发生变化了呢?

查找： QTextEdit中有 textChanged()信号

实现： 做信号与槽函数的关联

1.3.2.2 打开文件

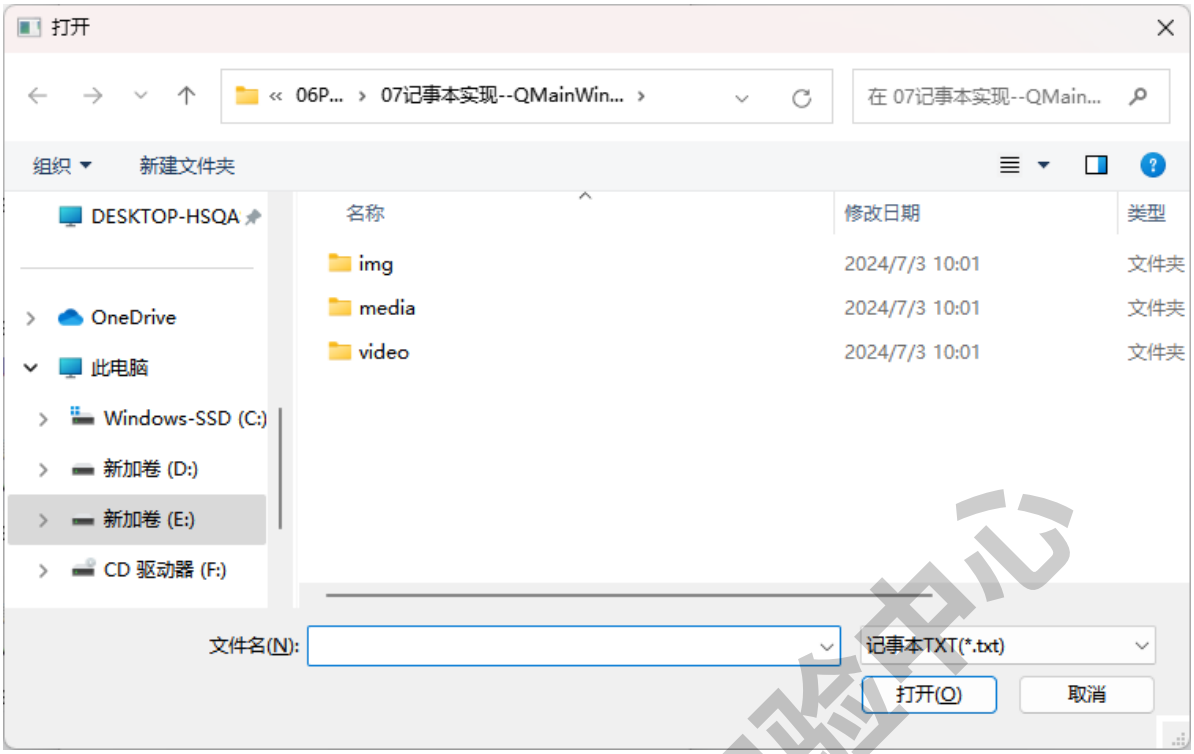
根据window中记事本的打开动作，

分析：

```
如果当前这个文本内容没有做任何修改：
    弹出一个文件选择对话框，选择一个文件并打开，读这个文件的内容，把读到内容设置到文本编辑框中，文件本身关闭！
    窗口标题变成了打开的这个文件的名字
如果当前这个文本内容有做修改：
    弹出消息对话框，询问是否对修改进行保存。
    保存：
        如果此时窗口标题为 无标题-记事本
            进行另存为
        否则：
            保存
        弹出一个文件选择对话框，选择一个文件并打开，读这个文件的内容，把读到内容设置到文本编辑框中，文件本身关闭！
        窗口标题变成了打开的这个文件的名字
    不保存：
        弹出一个文件选择对话框，选择一个文件并打开，读这个文件的内容，把读到内    容设置到文本编辑框中，文件本身关闭！
        窗口标题变成了打开的这个文件的名字
    取消：
        说明是误触，什么都不做
```

发现不管文本区域内容是否有修改，当前的文件是否保存，都会重复做一样的操作，那就是打开文件夹对话框，所以可以单独封装一个函数

华清远见元宇宙实验中心 yyzlab.com.cn



12-3-2-3、保存文件

根据window中记事本的保存动作，

分析：

```
如果当前操作的文件  文件名为   无标题-记事本
      进行另存为
否则：
      打开当前的这个文件，获取文本框中的内容，写入到文件中，文件关闭！
      窗口标题变成了当前打开文件的文件字
      修改文本框中的文本状态为 未修改
```

1.3.2.4 文件另存为

```
根据window中记事本的另存为动作，
分析：
      弹出一个文件选择对话框，输入新保存的文件名字，保存
      打开文件，获取文本框中的内容， 写入文件中， 文件关闭
      窗口标题变成新保存文件的名字！
      修改文本框中的文本状态为 未修改
代码实现：
```

1.3.2.5 文本的复制、粘贴、剪切、撤销、恢复

分析：

这些操作都是对文本框中的内容做相关操作

在QTextEdit中查找 都有对应的槽函数 copy()、paste()、cut()、undo()、redo()

代码实现：

华清远见元宇宙实验中心 yyzlab.com.cn

```
# 编辑菜单下动作的信号与槽函数的关联
self.copy_action.triggered.connect(self.text_edit.copy)
self.paste_action.triggered.connect(self.text_edit.paste)
self.cut_action.triggered.connect(self.text_edit.cut)
self.undo_action.triggered.connect(self.text_edit.undo)
self.redo_action.triggered.connect(self.text_edit.redo)
```

只要做action动作中的triggered信号与对应槽函数的关联即可。一旦有信号被触发，对应的槽函数自然被调用

1.3.3 工具栏业务实现

工具栏作用：用于提供快速访问应用程序的一些常用功能。

工具栏通常包含了一系列按钮、工具和选项，用户可以通过单击这些按钮来执行相应的操作。

例如： 字体选择、颜色选择等

2. 总结

在 PyQt5 中，`QFile` 是一个用于处理文件的类。它属于 `PyQt5.QtCore` 模块，主要用于文件的读取、写入和操作。`QFile` 提供了跨平台的文件操作功能，并且与 Qt 的其他模块（如 `QTextStream` 和 `QDataStream`）结合使用时，可以方便地进行文件的读写操作。

2.1 QFile

`QFile` 的主要作用是提供对文件的基本操作，包括：

- **打开文件**：以只读、只写或读写模式打开文件。
- **读取文件内容**：从文件中读取数据。
- **写入文件内容**：将数据写入文件。
- **关闭文件**：关闭已打开的文件。
- **检查文件状态**：检查文件是否存在、是否可读、是否可写等。
- **删除文件**：删除指定的文件。

2.1.1 `QFile` 的导入方式

```
from PyQt5.QtCore import QFile
```

2.1.2 常用方法

2.1.2.1 打开文件

- `open(QIODevice.OpenMode mode)`：以指定的模式打开文件。
  - 常用的打开模式有：
    - `QIODevice.ReadOnly`：只读模式。
    - `QIODevice.WriteOnly`：只写模式。
    - `QIODevice.ReadWrite`：读写模式。
    - `QIODevice.Append`：追加模式（在文件末尾写入数据）。
    - `QIODevice.Text`：文本模式（自动处理换行符）。

华清远见元宇宙实验中心 yyzlab.com.cn

```
file = QFile("example.txt")
if file.open(QIODevice.ReadOnly):
    print("文件打开成功")
```

2.1.2.2 读取文件内容

- `read(int maxSize)`：读取指定字节数的数据。

```
data = file.read(100)  # 读取最多 100 字节的数据
print(data)
```

- `readAll()`：读取文件中的所有数据。

```
data = file.readAll()  # 读取整个文件的内容
print(data)
```

- `readLine(int maxSize=0)`：读取一行数据（以换行符为分隔符）。

```
line = file.readLine()  # 读取一行数据
print(line)
```

2.1.2.3 写入文件内容

- `write(bytes data)`：将数据写入文件。

```
file.write(b"Hello, PyQt5!")  # 写入字节数据
```

- `putChar(str char)`：写入单个字符。

```
file.putChar('A')  # 写入单个字符 'A'
```

2.1.2.4 关闭文件

- `close()`：关闭已打开的文件。

```
file.close()
```

2.1.2.5 检查文件状态

- `exists()`：检查文件是否存在。

```
if QFile.exists("example.txt"):
    print("文件存在")
```

- `isReadable()`：检查文件是否可读。

```
if file.isReadable():
    print("文件可读")
```

- `isWritable()`：检查文件是否可写。

华清远见元宇宙实验中心 yyzlab.com.cn

```
if file.isWritable():
    print("文件可写")
```

- `size()`：获取文件的大小（以字节为单位）。

```
file_size = file.size()
print(f"文件大小: {file_size} 字节")
```

2.1.2.6 删除文件

- `remove()`：删除指定的文件。

```
if QFile.remove("example.txt"):
    print("文件删除成功")
```

2.1.2.7 重命名文件

- `rename(str newName)`：重命名文件。

```
if file.rename("new_example.txt"):
    print("文件重命名成功")
```

2.1.2.8 复制文件

- `copy(str newName)`：复制文件到新的位置。

```
if QFile.copy("example.txt", "copy_example.txt"):
    print("文件复制成功")
```

2.1.3 典型应用场景

- **读取配置文件**：可以从配置文件中读取应用程序的设置。
- **保存用户数据**：可以将用户的输入或应用程序的状态保存到文件中。
- **日志记录**：可以将程序运行时的日志信息写入文件。
- **文件管理**：可以执行文件的创建、删除、重命名等操作。

2.2 QFileInfo

在 PyQt5 中，`QFileInfo` 是一个用于获取文件或目录信息的类。它属于 `PyQt5.QtCore` 模块，主要用于查询文件或目录的属性，例如文件大小、创建时间、修改时间、权限等。`QFileInfo` 提供了一种跨平台的方式来获取文件系统的元数据。

`QFileInfo` 的主要作用是提供对文件或目录的详细信息的访问。它可以用于以下场景：

- **获取文件的基本信息**：如文件名、路径、扩展名等。
- **获取文件的时间信息**：如创建时间、修改时间、访问时间等。
- **检查文件的状态**：如文件是否存在、是否可读、是否可写、是否为目录等。
- **获取文件大小**：可以获取文件的大小（以字节为单位）。
- **处理文件路径**：可以解析文件路径中的各个部分，如绝对路径、相对路径等。

华清远见|元宇宙实验中心 yyzlab.com.cn

2.2.1 `QFileInfo` 的导入方式

```
from PyQt5.QtCore import QFileInfo
```

2.2.2 常用方法

2.2.2.1 创建 `QFileInfo` 对象

- `QFileInfo(str filepath)`：通过文件路径创建一个 `QFileInfo` 对象。

  ```
  file_info = QFileInfo("example.txt")
  ```

2.2.2.2 获取文件的基本信息

- `fileName()`：获取文件名（包括扩展名）。

  ```
  filename = file_info.fileName()
  print(f"文件名: {filename}")
  ```

- `baseName()`：获取文件的基本名称（不包括扩展名）。

  ```
  basename = file_info.baseName()
  print(f"基本名称: {basename}")
  ```

- `suffix()`：获取文件的扩展名。

  ```
  extension = file_info.suffix()
  print(f"扩展名: {extension}")
  ```

- `absoluteFilePath()`：获取文件的绝对路径。

  ```
  absolute_path = file_info.absoluteFilePath()
  print(f"绝对路径: {absolute_path}")
  ```

- `path()`：获取文件所在的目录路径。

  ```
  directory = file_info.path()
  print(f"目录路径: {directory}")
  ```

2.2.2.3 获取文件的时间信息

- `created()`：获取文件的创建时间。

  ```
  created_time = file_info.created()
  print(f"创建时间: {created_time.toString()}")
  ```

- `lastModified()`：获取文件的最后修改时间。

  ```
  modified_time = file_info.lastModified()
  print(f"最后修改时间: {modified_time.toString()}")
  ```

华清远见元宇宙实验中心
yyzlab.com.cn

- `lastRead()`：获取文件的最后访问时间。

```
read_time = file_info.lastRead()
print(f"最后访问时间: {read_time.toString()}")
```

2.2.2.4 检查文件的状态

- `exists()`：检查文件或目录是否存在。

```
if file_info.exists():
    print("文件存在")
```

- `isFile()`：检查是否为普通文件。

```
if file_info.isFile():
    print("这是一个文件")
```

- `isDir()`：检查是否为目录。

```
if file_info.isDir():
    print("这是一个目录")
```

- `isReadable()`：检查文件是否可读。

```
if file_info.isReadable():
    print("文件可读")
```

- `isWritable()`：检查文件是否可写。

```
if file_info.isWritable():
    print("文件可写")
```

- `isExecutable()`：检查文件是否可执行。

```
if file_info.isExecutable():
    print("文件可执行")
```

2.2.2.5 获取文件大小

- `size()`：获取文件的大小（以字节为单位）。

```
file_size = file_info.size()
print(f"文件大小: {file_size} 字节")
```

2.2.2.6 处理符号链接

- `isSymLink()`：检查文件是否为符号链接。

```
if file_info.isSymLink():
    print("这是一个符号链接")
```

华清远见元宇宙实验中心
yyzlab.com.cn

- `symLinkTarget()`：获取符号链接指向的目标文件。

```
target = file_info.symLinkTarget()
print(f"符号链接指向的目标: {target}")
```

2.2.3 典型应用场景

- **文件管理器**：可以使用 `QFileInfo` 来显示文件的详细信息，如文件名、大小、修改时间等。
- **文件监控**：可以定期检查文件的状态（如是否存在、是否被修改）。
- **日志记录**：可以记录文件的创建时间、修改时间等信息。
- **文件操作**：可以检查文件的权限、大小等信息，以便进行进一步的操作。

2.3 QKeySequence

在 PyQt5 中，`QKeySequence` 是一个用于表示键盘快捷键的类。它属于 `PyQt5.QtGui` 模块，主要用于定义和处理键盘快捷键组合（如 `Ctrl+S`、`Alt+F4` 等）。`QKeySequence` 提供了一种跨平台的方式来处理标准快捷键，并且可以轻松地与菜单项、工具栏按钮等控件关联。

`QKeySequence` 的主要作用是定义和处理键盘快捷键组合。它可以用于以下场景：

- **为菜单项或工具栏按钮设置快捷键**：例如，将 `Ctrl+S` 绑定到“保存”操作。
- **支持标准快捷键**：提供了许多预定义的标准快捷键（如 `QKeySequence.Save` 表示保存操作）。
- **解析用户输入的快捷键**：可以从字符串中解析快捷键组合。
- **跨平台支持**：确保快捷键在不同操作系统上的行为一致。

2.3.1 `QKeySequence` 的导入方式

```
from PyQt5.QtGui import QKeySequence
```

2.3.2 常用方法

2.3.2.1 创建快捷键

- `QKeySequence(str key)`：通过字符串创建快捷键组合。

```
shortcut = QKeySequence("Ctrl+S")  # 创建 Ctrl+S 快捷键
```

- `QKeySequence(int key)`：通过整数形式的按键值创建快捷键。

```
from PyQt5.QtCore import Qt

shortcut = QKeySequence(Qt.CTRL + Qt.Key_S)  # 使用整数形式创建 Ctrl+S 快捷键
```

- **使用标准快捷键**：
  - `QKeySequence.StandardKey` 提供了许多预定义的标准快捷键，例如：
    - `QKeySequence.Save`：保存操作。
    - `QKeySequence.Open`：打开操作。
    - `QKeySequence.Quit`：退出操作。
    - `QKeySequence.Copy`：复制操作。
    - `QKeySequence.Paste`：粘贴操作。

作清远见元宇宙实验中心 yyzlab.com.cn

```
save_shortcut = QKeySequence(QKeySequence.Save)  # 使用预定义的保存快捷键
```

2.3.2.2 获取快捷键的字符串表示

- `toString()`：将快捷键转换为字符串形式。

```
print(shortcut.toString())  # 输出 "Ctrl+S"
```

2.3.2.3 比较快捷键

- `matches(QKeySequence other)`：检查两个快捷键是否匹配。

```
if shortcut.matches(QKeySequence("Ctrl+S")):
    print("快捷键匹配")
```

2.3.2.4 绑定快捷键到 QAction

- `QAction.setShortcut(QKeySequence)`：为菜单项或工具栏按钮设置快捷键。

```
from PyQt5.QtWidgets import QAction

action = QAction("保存", None)
action.setShortcut(QKeySequence("Ctrl+S"))  # 设置快捷键为 Ctrl+S
```

2.3.3 典型应用场景

- **为菜单项或工具栏按钮设置快捷键**：例如，将 `Ctrl+S` 绑定到“保存”操作。
- **支持标准快捷键**：使用预定义的标准快捷键，确保与用户习惯一致。
- **动态解析快捷键**：从用户输入的字符串中解析快捷键组合。
- **跨平台支持**：确保快捷键在不同操作系统上的行为一致。

2.4 QColor

在 PyQt5 中，`QColor` 是一个用于表示和操作颜色的类。它属于 `PyQt5.QtGui` 模块，主要用于定义和处理颜色值。`QColor` 支持多种颜色表示方式（如 RGB、HSV、CMYK 等），并且可以方便地与 Qt 的其他模块（如 `QPalette`、`QBrush` 等）结合使用。

`QColor` 的主要作用是表示和操作颜色，通常用于以下场景：

- **设置控件的颜色**：如窗口背景色、按钮文本颜色等。
- **定义绘图颜色**：在绘图时使用 `QPainter` 设置画笔或画刷的颜色。
- **颜色转换**：可以在不同的颜色模型之间进行转换，例如 RGB、HSV 和 CMYK。
- **透明度支持**：可以为颜色设置透明度（Alpha 通道）。

2.4.1 `QColor` 的导入方式

```
from PyQt5.QtGui import QColor
```

华清远见云宇宙实验中心 yyzlab.com.cn

2.4.2 常用方法

2.4.2.1 创建颜色

- **通过 RGB 值创建颜色**:

```
color = QColor(255, 0, 0)  # 创建红色 (RGB: 255, 0, 0)
```

- **通过十六进制字符串创建颜色**:

```
color = QColor("#FF0000")  # 使用十六进制表示红色
```

- **通过颜色名称创建颜色**:

```
color = QColor("red")  # 使用颜色名称表示红色
```

- **通过 HSV 值创建颜色**:

```
color = QColor.fromHsv(0, 255, 255)  # 使用 HSV 表示红色
```

- **通过 CMYK 值创建颜色**:

```
color = QColor.fromCmyk(0, 100, 100, 0)  # 使用 CMYK 表示红色
```

2.4.2.2 获取颜色值

- **获取 RGB 值**:

```
red = color.red()      # 获取红色分量
green = color.green()  # 获取绿色分量
blue = color.blue()    # 获取蓝色分量
print(f"RGB: {red}, {green}, {blue}")
```

- **获取 Alpha 值（透明度）**:

```
alpha = color.alpha()  # 获取透明度值（0-255）
print(f"Alpha: {alpha}")
```

- **获取 HSV 值**:

```
hue = color.hue()          # 获取色调
saturation = color.saturation()  # 获取饱和度
value = color.value()      # 获取亮度
print(f"HSV: {hue}, {saturation}, {value}")
```

2.4.2.3 设置颜色值

- **设置 RGB 值**:

```
color.setRgb(0, 255, 0)  # 设置为绿色
```

华清远见元宇宙实验中心 yyzlab.com.cn

- **设置 Alpha 值（透明度）**：

```
color.setAlpha(128)  # 设置半透明
```

- **设置 HSV 值**：

```
color.setHsv(120, 255, 255)  # 设置为绿色（HSV 表示）
```

2.4.2.4 颜色转换

- **转换为十六进制字符串**：

```
hex_color = color.name()  # 转换为十六进制字符串（如 "#FF0000"）
print(hex_color)
```

- **转换为 RGB 字符串**：

```
rgb_string = color.getRgb()  # 返回 (R, G, B, A) 元组
print(rgb_string)
```

- **转换为 HSV 字符串**：

```
hsv_string = color.getHsv()  # 返回 (H, S, V, A) 元组
print(hsv_string)
```

2.4.2.5 颜色有效性检查

- **检查颜色是否有效**：

```
if color.isValid():
    print("颜色有效")
else:
    print("颜色无效")
```

2.4.3 典型应用场景

- **设置控件的颜色**：可以为窗口、按钮、文本等控件设置颜色。
- **绘图操作**：在绘图时使用 `QPainter` 设置画笔或画刷的颜色。
- **颜色选择器**：实现颜色选择器功能，允许用户选择颜色。
- **动态调整颜色**：根据用户输入或其他条件动态调整颜色。

2.5 QMainWindow

在 PyQt5 中，`QMainWindow` 是一个用于创建主应用程序窗口的类。它属于 `PyQt5.QtWidgets` 模块，提供了一个标准的主窗口框架，包含菜单栏、工具栏、状态栏、中心部件等组件。`QMainWindow` 是开发复杂 GUI 应用程序的基础，适合构建具有多个功能模块的应用程序。

`QMainWindow` 的主要作用是为应用程序提供一个标准化的主窗口结构。它通常用于以下场景：

- **创建主窗口**：作为应用程序的主要窗口，包含菜单栏、工具栏、状态栏等。
- **管理布局**：支持将不同的控件和功能模块组织到窗口的不同区域。
- **提供扩展性**：允许开发者轻松添加菜单项、工具栏按钮、停靠窗口（Dock Widgets）等功能。
- **支持多文档界面（MDI）**：可以通过 `QMdiArea` 实现多文档界面。

请访问元宇宙实验中心 yyzlab.com.cn

2.5.1 `QMainWindow` 的导入方式

```
from PyQt5.QtWidgets import QMainWindow
```

2.5.2 常用方法

2.5.2.1 设置中心部件

- `setCentralWidget(QWidget widget)`：设置主窗口的中心部件。

```
from PyQt5.QtWidgets import QTextEdit

text_edit = QTextEdit(self)
self.setCentralWidget(text_edit)  # 将文本编辑器设置为中心部件
```

2.5.2.2 添加菜单栏

- `menuBar()`：获取或创建主窗口的菜单栏。

```
menu_bar = self.menuBar()
file_menu = menu_bar.addMenu("文件")  # 添加“文件”菜单
```

2.5.2.3 添加工具栏

- `addToolBar(str title)`：添加一个新的工具栏。

```
tool_bar = self.addToolBar("工具栏")
tool_bar.addAction("保存")
```

2.5.2.4 添加状态栏

- `statusBar()`：获取或创建主窗口的状态栏。

```
status_bar = self.statusBar()
status_bar.showMessage("准备就绪")  # 在状态栏显示消息
```

2.5.2.5 添加停靠窗口

- `addDockWidget(Qt.DockWidgetArea area, QDockWidget dockwidget)`：添加一个停靠窗口。

```
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QDockWidget, QListWidget

dock = QDockWidget("列表", self)
dock.setWidget(QListWidget())  # 设置停靠窗口的内容
self.addDockWidget(Qt.LeftDockWidgetArea, dock)  # 将停靠窗口添加到左侧
```

华清远见|元宇宙实验中心
yyzlab.com.cn

2.5.2.6 设置窗口标题

- `setWindowTitle(str title)`：设置主窗口的标题。

```
self.setWindowTitle("我的应用程序")
```

2.5.2.7 调整窗口大小

- `resize(int width, int height)`：设置主窗口的大小。

```
self.resize(800, 600)  # 设置窗口大小为 800x600
```

2.5.3 典型应用场景

- **创建主窗口**：构建具有菜单栏、工具栏、状态栏的主窗口。
- **组织复杂布局**：通过中心部件、停靠窗口等功能模块化地组织界面。
- **实现多文档界面（MDI）**：使用 `QMdiArea` 管理多个子窗口。
- **动态更新界面**：通过状态栏显示实时信息，或通过工具栏提供快捷操作。

2.6 QMenuBar

在 PyQt5 中，`QMenuBar` 是一个用于创建菜单栏的类。它属于 `PyQt5.QtWidgets` 模块，主要用于在主窗口（如 `QMainWindow`）中添加菜单栏，并组织菜单项和子菜单。`QMenuBar` 提供了一种标准化的方式来定义应用程序的菜单结构，例如“文件”、“编辑”、“视图”等菜单。

`QMenuBar` 的主要作用是为应用程序提供菜单栏，通常用于以下场景：

- **组织功能模块**：将应用程序的功能划分为多个菜单（如“文件”、“编辑”、“帮助”等）。
- **添加菜单项**：在菜单中添加可点击的操作项（如“保存”、“打开”等）。
- **支持子菜单**：可以嵌套子菜单，实现更复杂的菜单结构。
- **快捷键支持**：为菜单项绑定快捷键，提升用户体验。
- **动态更新菜单**：根据程序状态动态启用或禁用菜单项。

2.6.1 `QMenuBar` 的导入方式

```
from PyQt5.QtWidgets import QMenuBar, QMainWindow
```

2.6.2 常用方法

2.6.2.1 获取或创建菜单栏

- `menuBar()`：在 `QMainWindow` 中获取或创建菜单栏。

```
menu_bar = self.menuBar()
```

2.6.2.2 添加菜单

- `addMenu(str title)`：添加一个新的菜单。

```
file_menu = menu_bar.addMenu("文件")  # 添加“文件”菜单
edit_menu = menu_bar.addMenu("编辑")  # 添加“编辑”菜单
```

华清远见开源实验中心 yyzlab.com.cn

2.6.2.3 添加菜单项

- `addAction(QAction action)`：向菜单中添加一个操作项。

```
save_action = QAction("保存", self)
file_menu.addAction(save_action)
```

- `addAction(str text)`：直接添加一个带有文本的操作项。

```
file_menu.addAction("退出")
```

2.6.2.4 添加分隔符

- `addSeparator()`：在菜单中添加一条分隔线。

```
file_menu.addSeparator()  # 在菜单项之间添加分隔线
```

2.6.2.5 添加子菜单

- `addMenu(QMenu menu)`：向菜单中添加一个子菜单。

```
recent_files_menu = file_menu.addMenu("最近的文件")  # 添加“最近的文件”子菜单
```

2.6.2.6 设置快捷键

- `setShortcut(str key)`：为菜单项设置快捷键。

```
save_action.setShortcut("Ctrl+S")  # 设置快捷键 Ctrl+S
```

2.6.2.7 响应菜单项操作

- `triggered.connect(slot)`：连接菜单项的触发信号到槽函数。

```
save_action.triggered.connect(self.save_file)
```

2.6.3 典型应用场景

- **组织功能模块**：通过菜单栏将应用程序的功能划分为多个菜单。
- **添加快捷键**：为常用操作绑定快捷键，提升用户效率。
- **动态更新菜单**：根据程序状态动态启用或禁用菜单项。
- **嵌套子菜单**：通过子菜单实现更复杂的菜单结构。

2.7 QMenu

在 PyQt5 中，`QMenu` 是一个用于创建菜单的类。它属于 `PyQt5.QtWidgets` 模块，主要用于组织和管理菜单项（如“文件”、“编辑”等菜单中的选项）。`QMenu` 通常与 `QMenuBar` 或右键上下文菜单结合使用，提供了一种灵活的方式来定义应用程序的功能结构。

`QMenu` 的主要作用是为应用程序提供菜单功能，通常用于以下场景：

- **组织菜单项**：将相关的操作分组到一个菜单中（如“文件”菜单下的“保存”、“打开”等）。
- **支持子菜单**：可以嵌套子菜单，实现更复杂的菜单结构。
- **快捷键支持**：为菜单项绑定快捷键，提升用户体验。

华清远见元宇宙实验中心 yyzlab.com.cn

- **动态更新菜单**：根据程序状态动态启用或禁用菜单项。
- **右键上下文菜单**：通过 `QMenu` 创建右键弹出菜单。

2.7.1 `QMenu` 的导入方式

```
from PyQt5.QtWidgets import QMenu, QAction
```

2.7.2 常用方法

2.7.2.1 创建菜单

- `QMenu(parent=None)`：创建一个新的菜单。

```
menu = QMenu("文件", self)
```

2.7.2.2 添加菜单项

- `addAction(QAction action)`：向菜单中添加一个操作项。

```
save_action = QAction("保存", self)
menu.addAction(save_action)
```

- `addAction(str text)`：直接添加一个带有文本的操作项。

```
menu.addAction("退出")
```

2.7.2.3 添加分隔符

- `addSeparator()`：在菜单中添加一条分隔线。

```
menu.addSeparator()  # 在菜单项之间添加分隔线
```

2.7.2.4 添加子菜单

- `addMenu(QMenu menu)`：向菜单中添加一个子菜单。

```
recent_files_menu = QMenu("最近的文件", self)
menu.addMenu(recent_files_menu)
```

2.7.2.5 设置快捷键

- `setShortcut(str key)`：为菜单项设置快捷键。

```
save_action.setShortcut("Ctrl+S")  # 设置快捷键 Ctrl+S
```

2.7.2.6 响应菜单项操作

- `triggered.connect(slot)`：连接菜单项的触发信号到槽函数。

```
save_action.triggered.connect(self.save_file)
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.7.2.7 作为右键上下文菜单

- `exec_(QPoint pos)`：在指定位置显示右键上下文菜单。

```
menu.exec_(self.mapToGlobal(event.pos()))
```

2.7.3 典型应用场景

- **组织功能模块**：通过菜单将应用程序的功能划分为多个逻辑组。
- **添加快捷键**：为常用操作绑定快捷键，提升用户效率。
- **动态更新菜单**：根据程序状态动态启用或禁用菜单项。
- **嵌套子菜单**：通过子菜单实现更复杂的菜单结构。
- **右键上下文菜单**：为控件提供右键弹出菜单。

2.8 QAction

在 PyQt5 中，`QAction` 是一个用于表示用户界面中的操作（如菜单项、工具栏按钮等）的类。它属于 `PyQt5.QtWidgets` 模块，主要用于定义和管理用户可以执行的操作。`QAction` 提供了一种统一的方式来定义操作，并将其与菜单项、工具栏按钮或其他控件关联起来。

`QAction` 的主要作用是为应用程序提供可复用的操作定义，通常用于以下场景：

- **菜单项**：将操作添加到菜单中（如“文件”菜单下的“保存”、“打开”等）。
- **工具栏按钮**：将操作添加到工具栏中，作为快捷按钮。
- **快捷键支持**：为操作绑定快捷键，提升用户体验。
- **图标支持**：为操作添加图标，增强视觉效果。
- **动态更新**：根据程序状态动态启用或禁用操作。

2.8.1 `QAction` 的导入方式

```
from PyQt5.QtWidgets import QAction
```

2.8.2 常用方法

2.8.2.1 创建操作.

- `QAction(str text, parent=None)`：通过文本创建一个操作。

```
save_action = QAction("保存", self)
```

- `QAction(QIcon icon, str text, parent=None)`：通过图标和文本创建一个操作。

```
from PyQt5.QtGui import QIcon

save_action = QAction(QIcon("save.png"), "保存", self)
```

2.8.2.2 设置快捷键

- `setShortcut(str key)`：为操作设置快捷键。

```
save_action.setShortcut("Ctrl+S")  # 设置快捷键 Ctrl+S
```

华清远见|元宇宙实验中心 yyzlab.com.cn

2.8.2.3 设置提示信息

- `setStatusTip(str tip)`：设置操作的状态栏提示信息。

```
save_action.setStatusTip("保存当前文件")
```

- `setToolTip(str tip)`：设置操作的工具提示信息（鼠标悬停时显示）。

```
save_action.setToolTip("点击以保存文件")
```

2.8.2.4 启用或禁用操作

- `setEnabled(bool enable)`：启用或禁用操作。

```
save_action.setEnabled(False)  # 禁用操作
```

2.8.2.5 连接信号与槽

- `triggered.connect(slot)`：连接操作的触发信号到槽函数。

```
save_action.triggered.connect(self.save_file)
```

2.8.2.6 检查操作是否被选中

- `setChecked(bool checked)`：设置操作的选中状态（适用于可切换的操作）。

```
save_action.setChecked(True)
```

- `isChecked()`：检查操作是否被选中。

```
if save_action.isChecked():
    print("操作已选中")
```

2.8.2.7 将操作添加到菜单或工具栏

- `addAction(QAction action)`：将操作添加到菜单或工具栏中。

```
file_menu.addAction(save_action)
tool_bar.addAction(save_action)
```

2.8.3 典型应用场景

- **菜单项**：将操作添加到菜单中，例如“文件”菜单下的“保存”、“打开”等。
- **工具栏按钮**：将操作添加到工具栏中，作为快捷按钮。
- **快捷键支持**：为常用操作绑定快捷键，提升用户效率。
- **动态更新**：根据程序状态动态启用或禁用操作。
- **可切换操作**：实现复选框式或单选按钮式的操作。

2.10 QToolButton

`QToolButton` 的主要作用是为应用程序提供工具栏按钮的功能，通常用于以下场景：

华清远见元宇宙实验中心 yyzlab.com.cn

- **工具栏按钮**：作为工具栏中的快捷按钮，执行特定的操作。
- **图标支持**：可以显示图标，并且支持多种图标模式（如仅显示图标或同时显示文本）。
- **下拉菜单**：可以附加一个下拉菜单，提供更多选项。
- **自动提升样式**：可以根据窗口风格自动调整按钮的外观。
- **切换按钮**：可以设置为可切换的状态（类似于复选框）。

2.10.1 `QToolButton` 的导入方式

```
from PyQt5.QtWidgets import QToolButton
```

2.10.2 常用方法

2.10.2.1 设置图标

- `setIcon(QIcon icon)`：为按钮设置图标。

```
from PyQt5.QtGui import QIcon

tool_button.setIcon(QIcon("icon.png"))
```

2.10.2.2 设置文本

- `setText(str text)`：为按钮设置文本。

```
tool_button.setText("保存")
```

2.10.2.3 设置工具提示

- `setToolTip(str tip)`：设置按钮的工具提示信息（鼠标悬停时显示）。

```
tool_button.setToolTip("点击以保存文件")
```

2.10.2.4 设置按钮样式

- `setToolButtonStyle(Qt.ToolButtonStyle style)`：设置按钮的显示样式。
  - `Qt.ToolButtonIconOnly`：仅显示图标（默认）。
  - `Qt.ToolButtonTextOnly`：仅显示文本。
  - `Qt.ToolButtonTextBesideIcon`：文本显示在图标旁边。
  - `Qt.ToolButtonTextUnderIcon`：文本显示在图标下方。

```
from PyQt5.QtCore import Qt

tool_button.setToolButtonStyle(Qt.ToolButtonTextUnderIcon)
```

2.10.2.5 设置自动提升样式

- `setAutoRaise(bool enable)`：启用或禁用自动提升样式（按钮在鼠标悬停时突出显示）。

```
tool_button.setAutoRaise(True)  # 启用自动提升样式
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.10.2.6 设置下拉菜单

- `setMenu(QMenu menu)`：为按钮设置一个下拉菜单。

```
from PyQt5.QtWidgets import QMenu, QAction

menu = QMenu(self)
action1 = QAction("选项1", self)
action2 = QAction("选项2", self)
menu.addAction(action1)
menu.addAction(action2)

tool_button.setMenu(menu)
```

- `setPopupMode(QToolButton.ToolButtonPopupMode mode)`：设置下拉菜单的弹出模式。
  - `QToolButton.MenuButtonPopup`：按钮右侧带有一个小箭头，点击箭头时显示菜单。
  - `QToolButton.InstantPopup`：点击按钮时直接显示菜单。
  - `QToolButton.DelayedPopup`：长按按钮时显示菜单。

```
tool_button.setPopupMode(QToolButton.MenuButtonPopup)
```

2.10.2.7 设置为切换按钮

- `setCheckable(bool checkable)`：将按钮设置为可切换状态。

```
tool_button.setCheckable(True)  # 设置为可切换状态
```

- `setChecked(bool checked)`：设置按钮的选中状态。

```
tool_button.setChecked(True)  # 设置为选中状态
```

2.10.2.8 连接信号与槽

- `clicked.connect(slot)`：连接按钮的点击信号到槽函数。

```
tool_button.clicked.connect(self.on_tool_button_clicked)
```

2.10.3 典型应用场景

- **工具栏按钮**：作为工具栏中的快捷按钮，执行特定的操作。
- **图标和文本组合**：根据需求选择不同的显示样式（仅图标、仅文本、图标+文本）。
- **下拉菜单**：通过下拉菜单提供更多选项。
- **切换按钮**：实现类似复选框的功能。
- **自动提升样式**：根据窗口风格动态调整按钮的外观。

2.11 QTextEdit

在 PyQt5 中，`QTextEdit` 是一个用于显示和编辑多行富文本或纯文本的控件。它属于 `PyQt5.QtWidgets` 模块，主要用于处理大段文本内容，支持文本格式化（如字体、颜色、对齐方式等）以及 HTML 格式的文本渲染。`QTextEdit` 提供了丰富的功能，适用于文本编辑器、日志查看器、HTML 渲染器等场景。

华清远见元宇宙实验中心 yyzlab.com.cn

`QTextEdit` 的主要作用是提供一个多行文本编辑区域，通常用于以下场景：

- **文本编辑**：允许用户输入和编辑多行文本。
- **富文本支持**：支持格式化的文本（如加粗、斜体、颜色、对齐方式等）。
- **HTML 渲染**：可以解析并显示 HTML 格式的文本。
- **只读模式**：可以用作只读的文本显示区域（如日志输出、帮助文档等）。
- **动态更新**：可以在运行时动态添加或修改文本内容。

2.11.1 `QTextEdit` 的导入方式

```
from PyQt5.QtWidgets import QTextEdit
```

2.11.2 常用方法

2.11.2.1 设置和获取文本

- `setText(str text)`：设置文本内容。

  ```
  text_edit.setText("这是一个示例文本")
  ```

- `toPlainText()`：获取纯文本内容。

  ```
  plain_text = text_edit.toPlainText()
  print(plain_text)
  ```

- `setHtml(str html)`：设置 HTML 格式的文本。

  ```
  text_edit.setHtml("<b>加粗文本</b> 和 <i>斜体文本</i>")
  ```

- `toHtml()`：获取 HTML 格式的文本。

  ```
  html_text = text_edit.toHtml()
  print(html_text)
  ```

2.11.2.2 插入文本

- `insertPlainText(str text)`：在光标位置插入纯文本。

  ```
  text_edit.insertPlainText("追加的文本")
  ```

- `insertHtml(str html)`：在光标位置插入 HTML 格式的文本。

  ```
  text_edit.insertHtml("<font color='red'>红色文本</font>")
  ```

2.11.2.3 设置只读模式

- `setReadOnly(bool ro)`：设置是否为只读模式。

  ```
  text_edit.setReadOnly(True)  # 设置为只读
  ```

华清远见元宇宙实验中心 yyzlab.com.cn

2.11.2.4 清空文本

- `clear()`：清空文本内容。

```
text_edit.clear()
```

2.11.2.5 文本格式化

- `setFont(QFont font)`：设置文本的字体。

```
from PyQt5.QtGui import QFont

font = QFont("Arial", 12)
text_edit.setFont(font)
```

- `setTextColor(QColor color)`：设置文本的颜色。

```
from PyQt5.QtGui import QColor

text_edit.setTextColor(QColor("blue"))
```

- `setAlignment(Qt.Alignment alignment)`：设置文本的对齐方式。

```
from PyQt5.QtCore import Qt

text_edit.setAlignment(Qt.AlignCenter)  # 居中对齐
```

2.11.2.6 操作光标

- `textCursor()`：获取当前的文本光标。

```
cursor = text_edit.textCursor()
```

- `setTextCursor(QTextCursor cursor)`：设置新的文本光标。

```
text_edit.setTextCursor(cursor)
```

2.11.2.7 滚动条控制

- `ensureCursorVisible()`：确保光标可见（自动滚动到光标位置）。

```
text_edit.ensureCursorVisible()
```

2.11.3 典型应用场景

- **文本编辑器**：实现多行文本编辑功能，支持格式化文本。
- **日志查看器**：用作只读的文本显示区域，动态追加日志信息。
- **HTML 渲染器**：解析并显示 HTML 格式的文本。
- **富文本编辑**：支持加粗、斜体、颜色、对齐等格式化操作。
- **动态更新内容**：在运行时动态添加或修改文本内容。

2.12 QFileDialog

华清远见元宇宙实验中心 yyzlab.com.cn

在 PyQt5 中，`QFileDialog` 是一个用于打开和保存文件的对话框类。它属于 `PyQt5.QtWidgets` 模块，主要用于与用户交互以选择文件或目录。`QFileDialog` 提供了标准的文件选择界面，并支持多种模式（如打开文件、保存文件、选择目录等），是开发文件操作功能的重要工具。

`QFileDialog` 的主要作用是为用户提供一个图形化的文件选择界面，通常用于以下场景：

- **打开文件**：让用户选择要打开的文件。
- **保存文件**：让用户指定保存文件的位置和名称。
- **选择目录**：让用户选择一个目录。
- **过滤文件类型**：通过文件过滤器限制用户只能选择特定类型的文件。
- **多文件选择**：支持同时选择多个文件。

2.12.1 `QFileDialog` 的导入方式

```
from PyQt5.QtWidgets import QFileDialog
```

2.12.2 常用方法

2.12.2.1 打开文件

- `getOpenFileName(parent=None, caption="", directory="", filter="")`：打开单个文件选择对话框。
  - 返回值：`(file_path, selected_filter)`，其中 `file_path` 是选中的文件路径，`selected_filter` 是用户选择的文件过滤器。

```
file_path, _ = QFileDialog.getOpenFileName(self, "打开文件", "", "文本文件 (*.txt);;所有文件 (*)")
if file_path:
    print(f"选中的文件: {file_path}")
```

- `getOpenFileNames(parent=None, caption="", directory="", filter="")`：打开多文件选择对话框。
  - 返回值：`(file_paths, selected_filter)`，其中 `file_paths` 是选中的文件路径列表。

```
file_paths, _ = QFileDialog.getOpenFileNames(self, "打开多个文件", "", "图片文件 (*.png *.jpg);;所有文件 (*)")
if file_paths:
    print(f"选中的文件: {file_paths}")
```

2.12.2.2 保存文件

- `getSaveFileName(parent=None, caption="", directory="", filter="")`：打开保存文件对话框。
  - 返回值：`(file_path, selected_filter)`，其中 `file_path` 是用户指定的文件路径。

```
file_path, _ = QFileDialog.getSaveFileName(self, "保存文件", "", "文本文件 (*.txt);;所有文件 (*)")
if file_path:
    print(f"保存的文件路径: {file_path}")
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.12.2.3 选择目录

- `getExistingDirectory(parent=None, caption="", directory="")`：打开目录选择对话框。
  - 返回值：选中的目录路径。

```
dir_path = QFileDialog.getExistingDirectory(self, "选择目录", "")
if dir_path:
    print(f"选中的目录: {dir_path}")
```

2.12.2.4 设置默认路径

- `setDirectory(str directory)`：设置对话框的初始目录。

```
dialog = QFileDialog()
dialog.setDirectory("/path/to/default/directory")
```

2.12.2.5 设置文件过滤器

- `setNameFilter(str filter)`：设置文件过滤器，限制用户只能选择特定类型的文件。

```
dialog = QFileDialog()
dialog.setNameFilter("图片文件 (*.png *.jpg);;文本文件 (*.txt)")
```

2.12.2.6 启用多文件选择

- `setFileMode(QFileDialog.FileMode mode)`：设置文件选择模式。
  - `QFileDialog.ExistingFile`：选择单个现有文件。
  - `QFileDialog.ExistingFiles`：选择多个现有文件。
  - `QFileDialog.Directory`：选择目录。

```
dialog = QFileDialog()
dialog.setFileMode(QFileDialog.ExistingFiles)  # 允许多文件选择
```

2.12.2.7 显示对话框

- `exec_()`：以模态方式显示对话框。

```
dialog = QFileDialog()
if dialog.exec_():
    print("用户确认选择")
```

2.12.3 典型应用场景

- **打开文件**：让用户选择要打开的文件（如加载配置文件、读取数据文件等）。
- **保存文件**：让用户指定保存文件的位置和名称（如保存文档、导出数据等）。
- **选择目录**：让用户选择一个目录（如设置工作目录、批量处理文件等）。
- **过滤文件类型**：通过文件过滤器限制用户只能选择特定类型的文件。
- **多文件选择**：支持同时选择多个文件（如批量上传图片、处理多个文件等）。

2.13 QFontDialog

华清远见宇宙实验中心 yyzlab.com.cn

在 PyQt5 中，`QFontDialog` 是一个用于选择字体的对话框类。它属于 `PyQt5.QtWidgets` 模块，主要用于与用户交互以选择字体（包括字体名称、大小、样式等）。`QFontDialog` 提供了一个标准的字体选择界面，方便用户选择所需的字体，并将其应用到应用程序中的文本控件或其他需要字体设置的地方。

`QFontDialog` 的主要作用是为用户提供一个图形化的字体选择界面，通常用于以下场景：

- **选择字体**：让用户选择字体名称、大小、样式（如加粗、斜体）等。
- **动态更新字体**：将用户选择的字体应用到文本控件（如 `QLabel`、`QTextEdit` 等）中。
- **获取字体信息**：获取用户选择的字体对象（`QFont`），以便进一步处理。
- **支持预览**：用户可以在对话框中实时预览所选字体的效果。

2.13.1 `QFontDialog` 的导入方式

```
from PyQt5.QtWidgets import QFontDialog
```

2.13.2 常用方法

2.13.2.1 打开字体选择对话框

- `getFont(ok, parent=None)`：打开字体选择对话框，并返回用户选择的字体和确认状态。
  - 参数：
    - `ok`：布尔值引用，表示用户是否确认选择。
  - 返回值：`(QFont font, bool ok)`，其中 `font` 是用户选择的字体对象，`ok` 表示用户是否点击了“确定”按钮。

```
font, ok = QFontDialog.getFont()
if ok:
    print(f"选中的字体: {font.family()}, 大小: {font.pointSize()}")
```

2.13.2.2 设置初始字体

- `setCurrentFont(QFont font)`：设置对话框的初始字体。

```
dialog = QFontDialog()
dialog.setCurrentFont(QFont("Arial", 12))
```

2.13.2.3 启用或禁用选项

- `setOption(QFontDialog.FontDialogOption option, bool on=True)`：启用或禁用特定选项。
  - 常用选项：
    - `QFontDialog.NoButtons`：隐藏“确定”和“取消”按钮。
    - `QFontDialog.ScalableFonts`：仅显示可缩放字体。
    - `QFontDialog.MonospacedFonts`：仅显示等宽字体。
    - `QFontDialog.ProportionalFonts`：仅显示比例字体。

```
dialog = QFontDialog()
dialog.setOption(QFontDialog.ScalableFonts, True)  # 仅显示可缩放字体
```

华清远见元宇宙实验中心 yyzlab.com.cn

3.4 显示对话框

- `exec_()`：以模态方式显示对话框。

```
dialog = QFontDialog()
if dialog.exec_():
    selected_font = dialog.currentFont()
    print(f"选中的字体: {selected_font.family()}")
```

2.13.3 典型应用场景

- **文本编辑器**：允许用户选择字体并应用到文本编辑区域。
- **自定义界面**：让用户选择字体并动态更新界面中的文本控件。
- **字体预览工具**：提供字体选择和预览功能。
- **配置文件**：保存用户的字体选择，以便下次启动时加载。

2.14 QFontDialog

在 PyQt5 中，`QFontDialog` 是一个用于选择字体的对话框类。它属于 `PyQt5.QtWidgets` 模块，主要用于与用户交互以选择字体（包括字体名称、大小、样式等）。`QFontDialog` 提供了一个标准的字体选择界面，方便用户选择所需的字体，并将其应用到应用程序中的文本控件或其他需要字体设置的地方。

`QFontDialog` 的主要作用是为用户提供一个图形化的字体选择界面，通常用于以下场景：

- **选择字体**：让用户选择字体名称、大小、样式（如加粗、斜体）等。
- **动态更新字体**：将用户选择的字体应用到文本控件（如 `QLabel`、`QTextEdit` 等）中。
- **获取字体信息**：获取用户选择的字体对象（`QFont`），以便进一步处理。
- **支持预览**：用户可以在对话框中实时预览所选字体的效果。

2.14.1 `QFontDialog` 的导入方式

```
from PyQt5.QtWidgets import QFontDialog
```

2.14.2 常用方法

2.14.2.1 打开字体选择对话框

- `getFont(ok, parent=None)`：打开字体选择对话框，并返回用户选择的字体和确认状态。
  - 参数：
    - `ok`：布尔值引用，表示用户是否确认选择。
  - 返回值：`(QFont font, bool ok)`，其中 `font` 是用户选择的字体对象，`ok` 表示用户是否点击了“确定”按钮。

```
font, ok = QFontDialog.getFont()
if ok:
    print(f"选中的字体: {font.family()}, 大小: {font.pointSize()}")
```

仅供参考 yyzlab.com.cn

2.14.2.2 设置初始字体

- `setCurrentFont(QFont font)`：设置对话框的初始字体。

```
dialog = QFontDialog()
dialog.setCurrentFont(QFont("Arial", 12))
```

2.14.2.3 启用或禁用选项

- `setOption(QFontDialog.FontDialogOption option, bool on=True)`：启用或禁用特定选项。
  - 常用选项：
    - `QFontDialog.NoButtons`：隐藏“确定”和“取消”按钮。
    - `QFontDialog.ScalableFonts`：仅显示可缩放字体。
    - `QFontDialog.MonospacedFonts`：仅显示等宽字体。
    - `QFontDialog.ProportionalFonts`：仅显示比例字体。

```
dialog = QFontDialog()
dialog.setOption(QFontDialog.ScalableFonts, True)  # 仅显示可缩放字体
```

2.14.2.4 显示对话框

- `exec_()`：以模态方式显示对话框。

```
dialog = QFontDialog()
if dialog.exec_():
    selected_font = dialog.currentFont()
    print(f"选中的字体: {selected_font.family()}")
```

2.14.3 典型应用场景

- **文本编辑器**：允许用户选择字体并应用到文本编辑区域。
- **自定义界面**：让用户选择字体并动态更新界面中的文本控件。
- **字体预览工具**：提供字体选择和预览功能。
- **配置文件**：保存用户的字体选择，以便下次启动时加载。

2.15 QColorDialog

在 PyQt5 中，`QColorDialog` 是一个用于选择颜色的对话框类。它属于 `PyQt5.QtWidgets` 模块，主要用于与用户交互以选择颜色（如背景色、文本颜色等）。`QColorDialog` 提供了一个标准的颜色选择界面，方便用户选择所需的颜色，并将其应用到应用程序中的控件或其他需要颜色设置的地方。

`QColorDialog` 的主要作用是为用户提供一个图形化的颜色选择界面，通常用于以下场景：

- **选择颜色**：让用户选择颜色（包括 RGB 值、HSV 值、Alpha 透明度等）。
- **动态更新颜色**：将用户选择的颜色应用到控件（如 `QLabel`、`QPushButton` 等）中。
- **获取颜色信息**：获取用户选择的颜色对象（`QColor`），以便进一步处理。
- **支持自定义颜色**：允许用户定义和保存自定义颜色。

华清远见|元宇宙实验中心 yvlab.com.cn

2.15.1 `QColorDialog` 的导入方式

```
from PyQt5.QtWidgets import QColorDialog
```

2.15.2 常用方法

2.15.2.1 打开颜色选择对话框

- `getColor(initial=None, parent=None, title="")`：打开颜色选择对话框，并返回用户选择的颜色。
  - 参数：
    - `initial`：初始颜色（默认为白色）。
    - `parent`：父窗口（可选）。
    - `title`：对话框标题（可选）。
  - 返回值：`QColor` 对象，表示用户选择的颜色。如果用户取消选择，则返回无效的颜色（`QColor.isValid()` 返回 `False`）。

```
color = QColorDialog.getColor()
if color.isValid():
    print(f"选中的颜色: {color.name()}")
```

2.15.2.2 设置初始颜色

- `setCurrentColor(QColor color)`：设置对话框的初始颜色。

```
dialog = QColorDialog()
dialog.setCurrentColor(QColor("red"))
```

2.15.2.3 启用或禁用选项

- `setOption(QColorDialog.ColorDialogOption option, bool on=True)`：启用或禁用特定选项。
  - 常用选项：
    - `QColorDialog.ShowAlphaChannel`：显示 Alpha 通道（透明度）。
    - `QColorDialog.NoButtons`：隐藏“确定”和“取消”按钮。
    - `QColorDialog.DontUseNativeDialog`：不使用系统原生对话框。

```
dialog = QColorDialog()
dialog.setOption(QColorDialog.ShowAlphaChannel, True)  # 显示透明度选项
```

3.4 显示对话框

- `exec_()`：以模态方式显示对话框。

```
dialog = QColorDialog()
if dialog.exec_():
    selected_color = dialog.currentColor()
    print(f"选中的颜色: {selected_color.name()}")
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.15.3 典型应用场景

- **设置控件颜色**：允许用户选择颜色并应用到控件（如背景色、文本颜色等）。
- **绘图工具**：让用户选择画笔或填充颜色。
- **主题配置**：让用户选择应用程序的主题颜色。
- **自定义颜色保存**：允许用户定义和保存自定义颜色。

华清远见|元宇宙实验中心
yyzlab.com.cn